Pedagogical tool for teaching counting from 1 to 6 in an interactive way

Misael Gallardo
Universidad de Atacama
Ingeniería Civil en ciencias de la
Computacion e Informatica
Copiapó, Chile
misael.gallardo.19@alumnos.uda
.cl

Abstract*—This paper presents the design and implementation of a four-state interactive counting system based on an Arduino Uno microcontroller, modeled as a Mealy finite-state machine. The system uses a single button to navigate between states, providing visual feedback through six LEDs to represent counts from 1 to 6. Development followed an activity-based methodology, dividing the logic into independent modules prior to full integration. The primary objective is to demonstrate the practical application of sequential logic and state-based control in embedded systems. The user interacts with the system through a button, generating input signals that are processed to control a set of LEDs, providing immediate visual feedback. A timing limitation related to the use of delay() was identified as an area for future improvement. This project also reinforces key concepts of automata theory and embedded system design through a modular and structured implementation approach.

Keywords*— Arduino, Mealy machine, logic architecture, finite-state machine, pedagogical tool, hardware interaction, modular development.

I. Introducción

Creación, planeación y desarrollo de una herramienta de aprendizaje de suma o conteo interactiva mediante uso de arduino uno, mediante una metodología de división de actividades para el aprendizaje superior de arquitectura lógica.

II. Metodología

Para asegurar el correcto funcionamiento del sistema, se identificaron los siguientes pasos clave para asegurar el correcto funcionamiento del sistema y permitir que su desarrollo fluyera sin mayores percances:

1) Definición de requisitos: En esta etapa, se estableció la necesidad de utilizar un botón como múltiple señal de entrada (“ON”/”OFF” y “Pulsación”).

2) Diseño del sistema: Se diseñó el sistema mediante división de actividades, en vez de trabajar con el sistema global en sí, pero siguiendo un esquema de estado básico planteado en un inicio (Ver imagen 1).

3) Desarrollo del código en Arduino: Se escribió el código según pequeñas actividades necesarias para el funcionamiento general (6 actividades).

4) Validación y pruebas: La realización de pruebas físicas para validar el correcto funcionamiento del sistema es vital, asegurando que el sistema responda apropiadamente. Además, se verifica que el tiempo de activación y el restablecimiento automático funcionaran de acuerdo a lo previsto.

Los principales componentes del sistema son tres:

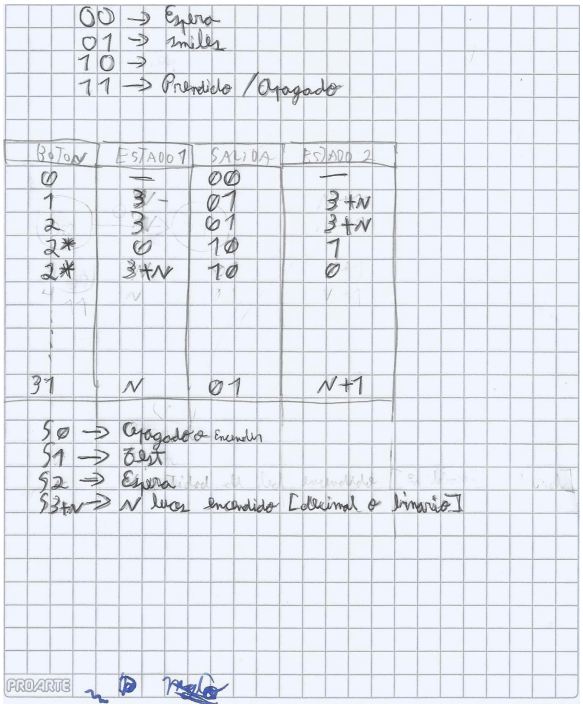
Microcontrolador Arduino Uno, que actúa como la unidad de procesamiento principal, gestionando las entradas y salidas del sistema.

Un botones (“A”), utilizados para ingresar la señal de cambio del sistema

Diodos emisores de luz (LEDs), que sirven como dispositivos visuales de avance, alerta y detención como sería en un sistema real.

Estos elementos, en conjunto, son los bloques fundamentales con los cuales se puede construir el sistema.

Imagen 1- Boceto inicial del sistema



Este boceto tuvo que ser delineado debido a que el lápiz gráfico no logra escanear correctamente.

III. Diseño del Sistema

El sistema está diseñado en torno a una lógica de pulso, donde las luces LEDs son apoyo visual del sistema, además de comprobación del correcto funcionamiento. El sistema es una herramienta que ayuda a contar según las veces que se presione un botón dando la retroalimentación con los LEDs. Este sistema tiene 4 estados. Para permitir un mejor entendimiento del sistema a desarrollar, es posible moldearlo como un autómata de estados finitos, particularmente como un autómata de Mealy, ya que las salidas dependen directamente de las transiciones entre estados y las entradas actuales, lo cual permite una respuesta más rápida y eficiente [1].

En este caso, el botón ("A") podrá dar 2 tipos de señales; 01 si el sistema está en un estado en concreto ($\mathbf{S_2}$) y que le permite contar las pulsaciones y encender luces LEDs según corresponda. 10 si el sistema está en estado de espera (misma señal para encender o apagar el sistema), la cual dará una señal diferente dependiendo del estado actual. Esta configuración trabaja con entrada de 2 bits (00, 01, 10), donde el principal objetivo es generar los estados necesarios para el correcto uso y entendimiento:

1) Estado inicial (S_0): El sistema espera el input del botón "A", ya que se encuentra en espera con todos los LEDs apagados, simulando estar apagado también. La señal de cambio de este sistema es mantener el "A" por 2 segundos continuamente.

2) Estado intermedio 1 (S_1): Si el estado anterior recibió la señal de cambio, el sistema pasará a un "Test" de LEDs para comprobar su correcto funcionamiento.

3) Estado intermedio 2 (S_2): Una vez que termina el "Test" el sistema queda en estado de espera por 5 segundos. Este estado espera el ingreso de señal del botón "A" para iniciar el conteo del 1 al 6, encendiendo un LED por cada vez que se presione. Si se presiona una vez llegado al 6, el sistema reinicia el conteo para volver a empezar. Hay 2 formas de salir de este estado; 1) Dejar que los 5 segundos pasen, llevando el sistema al siguiente estado. 2) Mantener el botón "A" por 2 segundos presionado, llevando al siguiente estado.

4) Estado Final (S_3): Este estado da una señal visual de que el sistema se está apagando.

Estos 4 estados están claramente definidos y cumplen un rol fundamental en el sistema.

La figura 1 presenta cada secuencia definida dentro del autómata de Mealy, donde la entrada del botón ("A") y la salida del sistema ("Z") son representados en el formato "A/Z". Si se presiona "A" mostrará como entrada "10/", y dará como resultado una salida "1" si pertenece a la entrada $\mathbf{S_0}$ y salida "0" si el tiempo de ingreso del input es erróneo (2 segundos), provocando que sea ignorado y espere otro ingreso. En caso que el botón no sea presionado "00/" la salida será "0", representando al sistema en un estado de espera de una entrada válida. En la tabla 1 puede apreciarse cómo las entradas dado un estado, da lugar al siguiente estado, dada las entradas estándar (00, 01, 10).

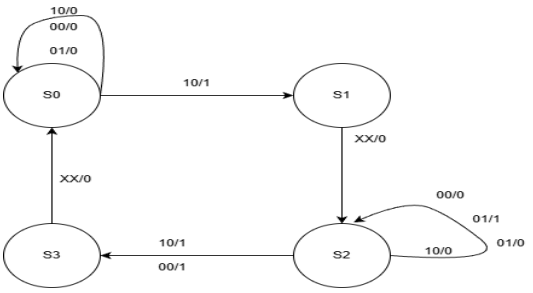


Fig. 1. Grafo del sistema mediante representación Mealy

TABLA I. Tabla de Estados Finitos.

Estado Actual	Entrada A	Entrada B	Estado siguiente	Salida
S0	0	0	S0	0
S0	0	1	S0	0
S0	1	0	S0	0
S0	1	0	S1	1
S1	X	X	S2	1
S2	0	0	S2	0
S2	0	0	S3	1
S2	0	1	S2	1
S2	1	0	S2	0
S2	1	0	S3	1
S3	X	X	S0	1

Las entradas no corresponden a señales binarias físicas simultáneas, sino a una abstracción lógica basada en duración y tipo de pulsación.La forma de enter la entradas en la siguiente:

Entrada 00 : Esta entrada indica que el botón no ha sido presionado. Se le considera entrada de espera.

Entrada 01 : Esta entrada indica que el botón fue presionado una vez rápidamente. Usado principalmente en el conteo del estado 2.

Entrada 10 : Esta entrada indica que se mantuvo presionando el botón por "x" cantidad de tiempo.

Entrada XX : Esta entrada indica que está activa cualquier entrada anterior.

Entrada 11 : Esta entrada no es usada debido que no hay planteamiento lógico ni físico que pueda representarla, dejando el sistema solo con 3 tipos de entradas necesarias.

La razón por la que una entrada pueda tener 2 salidas distintas es porque puede haber un falso positivo, un ejemplo es presionar el botón por 1 segundo, dando la entrada de 10, pero como no llega a los 2 segundos necesarios para el cambio de estado se considera falso positivo y expulsa una salida de 0. Otro ejemplo es la entrada 00 en el estado S_2 debido que dicha entrada es de espera para apagado donde espera 5 segundos para dar una salida de 1, y en caso que reciba una señal input saldrá una salida de 0 y reiniciará la espera.

El autómata de Mealy no es optimizable debido al formato de trabajo, donde se optimizo los estados antes de generar una tabla y un grafo representable

IV. Implementación

La implementación técnica de este proyecto se centró tanto en el desarrollo del código como en la integración de los componentes físicos, manteniendo la simplicidad del hardware gracias a la optimización de la lógica mediante el autómata de Mealy de la figura 1.

El microcontrolador Arduino Uno fue programado para manejar las entradas de un botón y procesarlo como una señal de cambio. La secuencia se dividió en las siguientes fases:

1) Inicialización del sistema: Se definen el estado inicial la variable necesaria para el botón (A) y los LEDs utilizados para el sistema. Se configuran los pines de entrada y salida del microcontrolador, y se inicializan los contadores y variables necesarias para gestionar la secuencia.

2) Lectura de entrada: El usuario presiona el botón, y según el modo el sistema hará una acción otra (presión rápida o mantención).

3) Acción de Encendido o Apagado: Si se recibe la señal (mantener el botón por 2 segundos) el sistema reconocerá el estado donde está y este procederá a encender o apagarse según corresponda.

4) Test: Si el sistema estaba en modo reposo (apagado), al encenderlo este comprobará que sus luces LED estén correctamente funcionando, siguiendo una secuencia de sus luces; llenado de conejo ascendente, descendente, recorrido de conejo normal y parpadeo de todas las luces.

Contador: El sistema se mantendrá en el estado $\mathbf{S_2}$ hasta que ocurra una de las 2 opciones de salida; presionar "A" por 2 segundos. Tiempo de espera superior de 5 segundos sin recibir input.

Aviso de Apagado: Una vez que el sistema quiere apagarse dará una señal parpadeando sus luces para luego volver a entrar a estado de reposo.

Con ello, el sistema debería ser capaz de manejar todos los estados disponibles del sistema, impidiendo estados no válidos y permitiendo una interacción con el usuario fluida, donde se comunica de manera efectiva los estados y funcionamiento de la máquina.

V. Coste de Producción

Dado que parte del objetivo de este proyecto, es producir físicamente el sistema, se detallan los costos de los componentes utilizados para el desarrollo, expresados en pesos chilenos:

TABLA 2. Precios de los Componentes

N°	Componentes necesarios para producir			
	Componente	**Cantidad**	**Coste Unitario**	**Costo Total**
1	Arduino Uno	1	$16.990	$16.990
2	Líneas Dupont	21	$250	$5.250
3	LEDs colores	11	$50	$550
4	Resistencia 1KΩ	9	$380	$3.420
5	Botones de 4 pines	1	$290	$290
Total				$26.500

La tabla 2 que se presenta muestra los costos de los componentes necesarios para el sistema. Sin embargo, es importante señalar que los precios que se indican se obtuvieron de mercados chinos o bazares electrónicos en los que se utilizaron modelos no oficiales, como unidades Arduino Uno sin marca. Estos precios tienden a ser más bajos que los de los modelos oficiales y no tienen en cuenta los gastos adicionales, como la mano de obra para el montaje, los derechos de importación y los costes de envío. Esto podría incurrir en un coste total considerablemente mayor.

Este análisis sirve como estimación del coste mínimo en circunstancias ideales, en lugar de ser un reflejo exhaustivo del coste real de fabricación e implementación del sistema.

VI. Resultados

El proyecto consiguió el resultado deseado, cumpliendo con todos los requisitos establecidos en un plazo óptimo. La demostración práctica y la implementación se completaron con éxito y el sistema funcionó como se esperaba, encendiendo y apagando con el su respectivo inpt de 2 segundos, comprobación de las luces LEDs mediante la función "Test" y correcto funcionamiento del estado de conteo. El sistema es capaz de esperar sus señales correspondientes en cada estado, además de realizar el encendido de los LEDs según las pulsaciones de "A".

Sin embargo, un problema notable emergió durante la etapa de pruebas. Dado el uso de funciones en el estado de "Test" llamando un total de 5 funciones que a veces por error del sobre llenado del espacio de memoria del arduino, donde las luces LEDs se comportan de manera errática ya por arquitectura física y lógica no debería error, o parpadeo irregular de un LEDs (Este comportamiento no debería ocurrir bajo condiciones normales de diseño, lo que sugiere posibles limitaciones en la gestión temporal o en la ejecución secuencial del sistema). Estos problemas ocurren en el momento de cambiar de funciones internas dentro del propio estado (Ver imagen 2), y el principal causante es la sentencia "delay()", que se debería cambiar por "millis()" en un futuro mantenimiento o arreglo. Otro desafío fue la validación del botón debido que este debe mantenerse presionado por un tiempo determinado y además ser capaz de contar las veces que es presionado evitando tener falsos positivos como presionar el botón antes de tiempo y luego volver a presión el tiempo faltante o que el programa contará de manera incorrecta el tiempo.

Imagen 2- código de la función Testeo de LEDs

```
void TEST(){
  parpadeo(5, 200);
  ascendente();
  ida();
  descendente();
  vuelta();
  conejo();
}
```

A pesar de este inconveniente, el comportamiento del sistema bajo condiciones normales es estable y confiable. El problema previamente mencionado otorga una valiosa perspectiva sobre el manejo del tiempo sobre el sistema, además del uso de memoria del arduino.

VII. Conclusión

En conclusión, mientras el sistema funciona como se esperaba bajo un uso típico, la limitación observada sirve como una experiencia importante. Enfatiza la importancia de arquitectura y control de los tiempos del sistema que tienen como propósito retroalimentar a los usuarios.

El proyecto aunque tomó más tiempo de lo esperado quizás por la metodología de arquitectura física, luego lógica para terminar en formato de estado y grafos complicó las cosas aunque facilitó el aprendizaje. El código (Arquitectura lógica) acabó siendo más complicado de lo esperado, debido a que al principio se trabajó de manera independiente cada situación (Ver imagen 3), por lo que unir todo en una sola sentencia fue desordenado, a pesar de tener las mismas variables iniciales. (Ver imagen 4).

Imagen 3- Muestra de actividad por separado del sistema

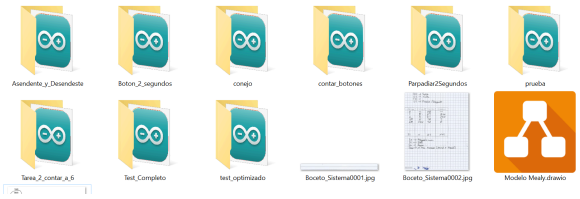


Imagen 4- Muestra de código de variables iniciales

```
Tarea_2_contar_a_6.ino
bool estadoBoton           = false;
int estado                 = 0;
const int led[]            = {4, 5, 6, 7, 8, 9};
const int nled             = 6;
unsigned long tiempoInicio = 0;
unsigned long tiempoFinal  = 0;
unsigned long tiempoApagado = 0;
const int boton            = 2;
int contador               = 0;
```

Aun así, este fue un proyecto divertido, un desarrollo didáctico que permite comprender mucho más allá el funcionamiento de los sistemas y aprender que el Arduino funciona de una manera concreta, y si no es así pues o no funciona, o crea falsos positivos dolorosos de corregir, en especial cuando la memoria se llena.

VIII. Referencias

[1] M. Melendez, “ARQUITECTURA DE SISTEMAS DIGITALES DE CONTROL: Un Enfoque Práctico”, pp. 6-11, no publicado